hashing algorithm to select the right node for a given key. By sharding, you can increase the capacity of your cache cluster.

As a rule of thumb: when a single node can no longer handle the amount of data or the requests, and if you are using Redis with mostly read traffic, then you should use replication. Replication also increases the availability at the same time (at no extra cost).

It is even possible to increase and decrease the number of shards automatically, based on utilization metrics, by using Application Auto Scaling. See http://mng.bz/82d2 to learn more.

Last but not least, replicating data is possible not only within a cluster but also between clusters running in other regions of the world. For example, if you are operating a popular online game, it is necessary to minimize latency by deploying your cloud infrastructure to the United States, Europe, and Asia. By using ElastiCache Global Datastore, you are able to replicate a Redis cluster located in Ireland to North Virginia and Singapore, for example.

11.6.3 Compressing your data

Instead of sending large values (and also keys) to your cache, you can compress the data before you store it in the cache. When you retrieve data from the cache, you have to uncompress it on the application before you can use the data. Depending on your data, compressing data can have a significant effect. We saw memory reductions to 25% of the original size and network transfer savings of the same size. Please note that this approach needs to be implemented in your application.

As a rule of thumb: compress your data using an algorithm that is best suited for your data. Most likely the `zlib` library is a good starting point. You might want to experiment with a subset of your data to select the best compression algorithm that is also supported by your programming language.

On top of that, ElastiCache for Redis also supports data tiering to reduce costs. With data tiering enabled, ElastiCache will move parts of your data from memory to solid-state disks—a tradeoff between costs and latency. Using data tiering is most interesting if your workload reads only parts of the data frequently.